

Índice

Oktubre Fest – (🔗 try-1)	1
Arrancar la primera parte	1
Mecanismo de decisión: herencia vs strategy	1
Template method	2
Template Method (🔗 refactor-template-method)	3

Oktubre Fest – (🔗 [try-1](#))

Los paquetes existen para agrupar clases relacionadas. No tiene sentido crear un paquete con una sola clase (ej: `pais/Pais.java`). Solo lo justificamos si el concepto crece y aparecen clases nuevas (ej: `Ciudad`, `Provincia`). Caso contrario, ponemos la clase en el paquete raíz y ahorramos ruido.

Arrancar la primera parte

Leemos la primera parte del enunciado: marcas, jarras. Ahí nomás tenemos que decidir cómo modelar las marcas y las jarras. Reflexionamos y nos damos cuenta que `Marca` debe ser una clase abstracta de donde salen marcas concretas.

Podemos dibujar las dependencias de lo que se ve en la parte 1 del enunciado.

```
Persona (depende de Jarra, Marca)
  ↑
Jarra (depende de Marca)
  ↑
Marca (depende de nada – en principio Pais podría ser un String, no una
clase)
```

En la parte 1, `Pais` solo es un atributo (un `String`). No lo modelamos como clase separada porque el dominio no lo exige todavía. Arrancamos por `Marca`, después `Jarra`, después `Persona`. Cada paso se para sobre el anterior y lo testeamos antes de avanzar.

Nos damos cuenta en el proceso de codeo que `Marca` debe tener un método abstracto para calcular la graduación.

Mecanismo de decisión: herencia vs strategy

Cuando nos encontramos con un comportamiento que varía, nos preguntamos:

“Esto que varía, ¿es algo que el objeto ES o algo que el objeto HACE?”

- Si es algo que el objeto **es** (una Rubia es una Marca, una Negra es una Marca) ⇒ va bien con herencia.
- Si es algo que el objeto **hace** (una Carpa calcula un recargo, una Persona decide si le gusta una marca) ⇒ nos preguntamos: ¿esto podría necesitar cambiar sin

reemplazar el objeto? ¿Podría haber formas distintas de hacerlo según el contexto? Ahí Strategy.

La herencia ata el comportamiento al tipo del objeto para siempre. Strategy nos deja cambiarlo en el momento que queramos, sin crear un objeto nuevo.

Hay dos tipos de «is-a»:

	Herencia de clase (extends)	Herencia de interfaz (implements)
Qué comparte	Estado + comportamiento	Solo contrato (métodos)
Ej	Rubia extends Marca	Pato implements Nadador
Significado	“es un tipo de”	“es capaz de”

El ejemplo del profesor (**Pato implements Nadador**) sigue siendo is-a: el pato **es capaz de** nadar. No es Strategy. Es polimorfismo con interfaces, pero no es el patrón Strategy.

Strategy es otra cosa: es **has-a** (composición). La persona no implementa **LeGustaStrategy**. La persona **tiene** un objeto que implementa **LeGustaStrategy**.

Los tres patterns usan interfaces, pero la relación es distinta:

Patrón	Relación
Herencia (class/subclass)	is-a $\Rightarrow$ extends
Interfaz (Pato/Nadador)	is-a (capaz de) $\Rightarrow$ implements
Strategy	has-a $\Rightarrow$ atributo de tipo interfaz

La confusión vino porque te mostré una interfaz (**LeGustaStrategy**) pensando en Strategy, y tu profesor te mostró otra interfaz (**Nadador**) pensando en polimorfismo con herencia de interfaz. Son dos usos distintos de interfaces.

Template method

En nuestro código actual, **no** es exactamente Template Method. Tanto **CervezaNegra** como **CervezaRoja** redefinen **graduacion()** enteras. Cada una tiene su propia implementación completa. **CervezaRoja** llama a **super.graduacion()** como un atajo, pero es una decisión de ella, no un diseño del padre.

En Template Method clásico, el **padre** tiene un método **concreto** (no abstracto) que es la plantilla, y llama adentro a métodos abstractos o sobrescribibles que las subclases implementan.

Nosotros tenemos

```
public class CervezaNegra extends Marca {
    //lógica

    @Override
    public Double graduacion() {
        // Al final dividimos por 100 para obtener la el porcentaje en formato
        decimal (entre 0 y 1).
        return (
            Math.min(
                // Para comparar estas magnitudes, lo que tiene sentido es que la
                graduación reglamentaria esté en porcentaje (multiplicada por
                100).
                CervezaNegra.graduacionReglamentaria * 100,
                2 * this.gramosLupulo
            ) / 100
        );
    }
}
```

Y algo similar para la clase CervezaRoja. Entonces no tenemos un Padre que gestiona.

- **Lo que tenemos:** cada hijo redefine `graduacion()` entero. El padre no controla nada.
- **Template Method:** el padre tiene un método plantilla concreto (`graduacion()`) que define el esqueleto, y los hijos solo modifican pasos específicos (`calcularGraduacionBase()`) sin reescribir la estructura completa.

En principio en este caso no hay problema con que sea así nomás. Es simple y funciona. Template Method tendría más sentido si el cálculo de graduación tuviera varios pasos y quisiéramos que algunos varíen y otros no.

Igualmente implementaremos Template Method en la rama [🔗 refactor-template-method](#).

Template Method ([🔗 refactor-template-method](#))

Fuimos a `Marca.java` y refactorizamos el método `graduacion()` para que se transforme en Template Method.

```
public abstract class Marca {  
    // lógica  
  
    // Template method. Método concreto que define el esqueleto y llama a  
    pasos internos.  
    public Double graduacion() {  
        return calcularGraduacion(); // <- llama a un paso  
    }  
  
    protected abstract Double calcularGraduacion(); // <- el paso que varía.  
}
```

Luego cada cerveza sobrescribe `calcularGraduacion()` para implementar su propia lógica. Con un solo paso, Template Method es puramente ceremonial. Es lo mismo que tener `graduacion()` abstracto directamente. La diferencia es conceptual, no práctica. El patrón muestra su valor cuando el esqueleto tiene **varios pasos**, algunos fijos y otros variables.